$\dot{H}$ Derivation

Sugheerth Sreedharan

October 4, 2025

1 Derivation of Derivative of Softmin

$$h^i = h^i_{\min_j} - \delta \ln\left(\frac{1}{N}\sum_{j=1}^{N} e^{-\hat{h}^i_j/\delta}\right)$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\delta}{N} \cdot \frac{\sum e^{-\hat{h}^i_j/\delta}}{\frac{1}{N}\sum e^{-\hat{h}^i_j/\delta}} \cdot \frac{-1}{\delta} \cdot (\dot{h^i_j} - \dot{h^i_{min_j}})$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}} - \dot{h^i_{\min_j}}\frac{\sum e^{-\hat{h}^i_j/\delta}}{\sum e^{-\hat{h}^i_j/\delta}}$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}} - \dot{h^i_{\min_j}}$$

$$\Rightarrow \quad \dot{h^i} = \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}}$$

2 Derivation of $\dot{h}^i_j$ in terms of Jacobians and $\dot{\mathbf{q}}$

The scalar function is given by:

$$\mathbf{\dot{h}^i_j(q_i)} = (\mathbf{X_j} - \mathbf{C_i})^{\mathbf{T}}\mathbf{R_i}\mathbf{D}\mathbf{R_i^T}(\mathbf{X_j} - \mathbf{C_i}) - \mathbf{1}$$

where $\mathbf{C_i} = \mathbf{C_i}(\mathbf{q_i}), \mathbf{R_i} = \mathbf{R_i}(\mathbf{q_i})$, and $\mathbf{q}_i = \mathbf{q}_{1:i}$ is the vector of joint angles up to link i. X_j is a constant point.

The time derivative $\mathbf{\dot{h}^i_j}$ is calculated using the product rule:

$$\dot{h}^i_j = \frac{d}{dt}\left((X_j - C_i)^T R_i D R_i^T (X_j - C_i) - 1\right)$$

$$\Longrightarrow \frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j} - \mathbf{C_i})^{\mathbf{T}}\right) R_i D R_i^T (X_j - C_i) \tag{1}$$

$$+ \quad (X_j - C_i)^T \frac{\mathbf{d}}{\mathbf{dt}}(\mathbf{R_i}) D R_i^T (X_j - C_i) \tag{2}$$

$$+ \quad (X_j - C_i)^T R_i D \frac{\mathbf{d}}{\mathbf{dt}}\left(\mathbf{R_i^T}\right)(X_j - C_i) \tag{3}$$

$$+ \quad (X_j - C_i)^T R_i D R_i^T \frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j} - \mathbf{C_i})\right) \tag{4}$$

2.1 Taking the terms (1) and (4) (Translation)

$$\frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j}-\mathbf{C_i})^{\mathbf{T}}\right)R_iDR_i^T(X_j-C_i)+(X_j-C_i)^TR_iDR_i^T\frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j}-\mathbf{C_i})\right)$$

Since X_j is constant, $\frac{d}{dt}(X_j)=\mathbf{0}$, which simplifies to:

$$\left(-\dot{\mathbf{C}}_{\mathbf{i}}^{\mathbf{T}}\right)R_iDR_i^T(X_j-C_i)+(X_j-C_i)^TR_iDR_i^T\left(-\dot{\mathbf{C}}_{\mathbf{i}}\right) \tag{5}$$

Both terms are scalars. Using the property $A=A^T$ where $A=(X_j-C_i)^TR_iDR_i^T\left(-\dot{\mathbf{C}}_{\mathbf{i}}\right)$:

$$-2\dot{C}_i^TR_iDR_i^T(X_j-C_i) \tag{6}$$

The linear velocity $\dot{\mathbf{C}}_{\mathbf{i}}$ is given by the linear Jacobian J_v:

$$\dot{C}_i=J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i \tag{7}$$

Substituting (7) in (6) gives:

$$-2\left(J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i\right)^TR_iDR_i^T(X_j-C_i) \tag{8}$$

2.2 Taking terms (2) and (3) (Rotation)

$$(X_j-C_i)^T\left(\dot{\mathbf{R}}_{\mathbf{i}}\right)DR_i^T(X_j-C_i)+(X_j-C_i)^TR_iD\left(\dot{\mathbf{R}}_{\mathbf{i}}^{\mathbf{T}}\right)(X_j-C_i) \tag{9}$$

Using $\dot{R}_i=[w_i]_xR_i$ and $\dot{R}_i^T=-R_i^T[w_i]_x$, and simplifying using the scalar property as before:

$$\begin{aligned}&(X_j-C_i)^T([\mathbf{w_i}]_\mathbf{x}\mathbf{R_i})DR_i^T(X_j-C_i)+(X_j-C_i)^TR_iD\left(-\mathbf{R_i^T}[\mathbf{w_i}]_\mathbf{x}\right)(X_j-C_i)\\ \implies\ &2(X_j-C_i)^T[\mathbf{w_i}]_\mathbf{x}\mathbf{R_i}DR_i^T(X_j-C_i)\end{aligned}$$

Using the property $\mathbf{a}^T([w]_x\mathbf{b})=w^T(\mathbf{b}\times\mathbf{a})$:

$$\implies 2w_i^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right) \tag{10}$$

The angular velocity w_i is given by the angular Jacobian J_w:

$$w_i=J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i \tag{11}$$

Substituting (11) in (10) gives:

$$2(J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right) \tag{12}$$

2.3 Combined Result

Combining (8) and (12):

$$\dot{h}_j^i=2[(J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-(J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^TR_iDR_i^T(X_j-C_i)]$$

Factoring out $\dot{\mathbf{q}}_i^T$:

$$\dot{h}_j^i=2\dot{\mathbf{q}}_i^T\left[J_w^T(\mathbf{q}_i)\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-J_v^T(\mathbf{q}_i)R_iDR_i^T(X_j-C_i)\right]$$

3 Final Combined result in softmin

The individual constraint time derivative $\dot{h}_j^i$ can be expressed as a linear product:

$$\dot{h}_j^i=\mathbf{A}_j^T\dot{\mathbf{q}}_i \tag{13}$$

where $\mathbf{A}_j^T$ is the Jacobian row vector for the j-th point:

$$\mathbf{A}_j^T=2\left[J_w^T(\mathbf{q}_i)\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-J_v^T(\mathbf{q}_i)R_iDR_i^T(X_j-C_i)\right] \tag{14}$$

The final weighted time derivative $\dot{h}^i$ is defined as:

$$\dot{h}^i = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \dot{h}^i_j}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{15}$$

Substituting $\dot{h}^i_j = \mathbf{A}_j^T \dot{\mathbf{q}}_i$ into the weighted average:

$$\dot{h}^i = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, (\mathbf{A}_j^T \dot{\mathbf{q}}_i)}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{16}$$

Since the joint velocity vector $\dot{\mathbf{q}}_i$ is common to all terms in the summation, it can be factored out:

$$\dot{h}^i = \left(\frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \mathbf{A}_j^T}{\sum_j e^{-\hat{h}^i_j/\delta}} \right) \dot{\mathbf{q}}_i \tag{17}$$

This simplifies to the desired linear form:

$$\dot{h}^i = \mathbf{J}^i_h \dot{\mathbf{q}}_i \tag{18}$$

where $\mathbf{J}^i_h$ is the **Combined Jacobian Row Vector** for the weighted constraint $\dot{h}^i$:

$$\mathbf{J}^i_h = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \mathbf{A}_j^T}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{19}$$

$$\dot{\tau}^j = \lim_{\Delta\tau \to 0} \frac{\tau^j - \tau^{j+1}}{\Delta\tau} \tag{20}$$

$$\dot{\tau}^j = u^j \tag{21}$$

$$h(u^j_k \mid x^j_k) := \frac{dH_\phi(x^j_k)}{dx^j_k}\, u^j_k + \alpha\big(H_\phi(x^j_k)\big) \geq 0 \tag{22}$$

4 The CBF of X_t

Notation

Let $X_t = [w_1, \ldots, w_T]^\top \in R^{2T}$, with each waypoint $w_i = (x_i, y_i)$. Define the squared signed distance from waypoint i to obstacle j as:

$$d_{ij} = (p_{xj} - x_i)^2 + (p_{yj} - y_i)^2 - r^2 + \gamma\delta \tag{23}$$

To focus gradient contributions on violated constraints ($d_{ij} < 0$) while suppressing safe ones ($d_{ij} > 0$), we apply a smooth asymmetric transformation via the softplus function:

$$\tilde{d}_{ij} = -c \cdot \text{softplus}\left(-\frac{d_{ij}}{c}\right) + log(2) = -c \cdot \log\left(1 + e^{-d_{ij}/c}\right) + log(2) \tag{24}$$

where $c > 0$ is a sharpness parameter. This transformation satisfies:

- $d_{ij} \gg 0 \;\Rightarrow\; \tilde{d}_{ij} \approx log(2)$ (safe obstacles contribute nothing)
- $d_{ij} \ll 0 \;\Rightarrow\; \tilde{d}_{ij} \approx d_{ij}$ (violated obstacles pass through unchanged)
- $\tilde{d}_{ij} \leq 0$ for all d_{ij}, with smooth transition at zero

The waypoint-level CBF uses a softmin over the transformed distances with temperature k_1:

$$h(w_i) = \text{softmin}^{k_1}_j(\tilde{d}_{ij}) = -k_1 \log \frac{1}{N} \sum_{j=1}^{N} e^{-\tilde{d}_{ij}/k_1} \tag{25}$$

The trajectory-level CBF uses a softmin over waypoints with temperature k_2:

$$h(X_t) = \text{softmin}^{k_2}_i(h(w_i)) = -k_2 \log \frac{1}{T} \sum_{i=1}^{T} e^{-h(w_i)/k_2} \tag{26}$$

Step 1 — Outer Softmin Weights (over waypoints, temperature k_2)

To ensure numerical stability when evaluating the exponentials, we factor out the minimum waypoint CBF value. Let $h_{\min} = \min_{i'} h(w_{i'})$.

The stabilized trajectory-level CBF is:

$$h(X_t) = h_{\min} - k_2 \log\left(\frac{1}{T}\sum_{i=1}^{T} e^{-(h(w_i)-h_{\min})/k_2}\right) \tag{27}$$

Define the stabilized softmax weights over waypoints:

$$\sigma_i = \frac{e^{-(h(w_i)-h_{\min})/k_2}}{\sum_{i'=1}^{T} e^{-(h(w_{i'})-h_{\min})/k_2}}, \qquad \sum_{i=1}^{T}\sigma_i = 1 \tag{28}$$

By the chain rule, the partial derivative of $h(X_t)$ with respect to waypoint w_i remains algebraically identical, now utilizing the stabilized weights:

$$\frac{\partial h(X_t)}{\partial w_i} = \sigma_i \cdot \nabla_{w_i} h(w_i) \tag{29}$$

Step 2 — Inner Softmin Weights (over obstacles, temperature k_1)

Similarly, we stabilize the waypoint-level CBF against the most deeply penetrated obstacle. Let $\tilde{d}_{\min,i} = \min_{j'} \tilde{d}_{ij'}$.

The stabilized waypoint-level CBF is:

$$h(w_i) = \tilde{d}_{\min,i} - k_1 \log\left(\frac{1}{N}\sum_{j=1}^{N} e^{-(\tilde{d}_{ij}-\tilde{d}_{\min,i})/k_1}\right) \tag{30}$$

Define the stabilized softmax weights over obstacles for waypoint i:

$$\alpha_{ij} = \frac{e^{-(\tilde{d}_{ij}-\tilde{d}_{\min,i})/k_1}}{\sum_{j'=1}^{N} e^{-(\tilde{d}_{ij'}-\tilde{d}_{\min,i})/k_1}}, \qquad \sum_{j=1}^{N}\alpha_{ij} = 1 \tag{31}$$

Since $\tilde{d}_{ij} \le \log(2)$ for all i, j, and the shift guarantees the largest exponent is exactly 0, all exponential terms are bounded in $(0, 1]$, preventing numerical overflow.

Differentiating $h(w_i)$ via the chain rule yields the same weighted sum, as the constant shift $\tilde{d}_{\min,i}$ vanishes during differentiation:

$$\nabla_{w_i} h(w_i) = \sum_{j=1}^{N} \alpha_{ij} \cdot \nabla_{w_i} \tilde{d}_{ij} \tag{32}$$

To compute $\nabla_{w_i}\tilde{d}_{ij}$, we apply the chain rule through the softplus. Since the derivative of softplus is the sigmoid $\sigma(\cdot) = (1 + e^{-(\cdot)})^{-1}$, and the $+\log(2)$ offset is constant:

$$\frac{\partial \tilde{d}_{ij}}{\partial d_{ij}} = \sigma\left(-\frac{d_{ij}}{c}\right) = \frac{1}{1 + e^{d_{ij}/c}} \tag{33}$$

Combined with $\dfrac{\partial d_{ij}}{\partial w_i} = -2(p_j - w_i)$, we obtain:

$$\nabla_{w_i}\tilde{d}_{ij} = -2\,\sigma\left(-\frac{d_{ij}}{c}\right)(p_j - w_i) \tag{34}$$

Substituting back:

$$\boxed{\nabla_{w_i} h(w_i) = -2 \sum_{j=1}^{N} \alpha_{ij}\, \sigma\!\left(-\frac{d_{ij}}{c}\right)(p_j - w_i)} \tag{35}$$

The sigmoid factor $\sigma(-d_{ij}/c)$ acts as a **soft gate**:

- $d_{ij} \ll 0$: $\sigma(-d_{ij}/c) \to 1$ (violated obstacles pass full gradient)
- $d_{ij} \gg 0$: $\sigma(-d_{ij}/c) \to 0$ (safe obstacles contribute no gradient)
- $d_{ij} = 0$: $\sigma(0) = 0.5$ (smooth transition at the safety boundary)

Step 3 — Full Gradient

Since the waypoints $\{w_i\}$ are independent components of X_t, the full gradient is assembled blockwise, utilizing the numerically stabilized weights σ_i and α_{ij} derived above:

$$\boxed{\nabla_{X_t} h(X_t) = \left\{ -2\,\sigma_i \sum_{j=1}^{N} \alpha_{ij}\, \sigma\!\left(-\frac{d_{ij}}{c}\right) \begin{pmatrix} p_{xj} - x_i \\ p_{yj} - y_i \end{pmatrix} \right\}_{i=1}^{T}} \tag{36}$$

This is a vector in $R^{T\times 2}$, where the block for waypoint w_i is a **triply-weighted** sum of displacement vectors from w_i to each obstacle center p_j.

Summary of Weights

Symbol	Parameter	Role
σ_i	k_2	Importance of waypoint i to trajectory-level CBF
α_{ij}	k_1	Importance of obstacle j to waypoint-level CBF
$\sigma(-d_{ij}/c)$	c	Gate suppressing safe obstacles from gradient

Effect of parameters.

- $k_1 \to 0$: α_{ij} concentrates on the single most-violated obstacle.
- $k_2 \to 0$: σ_i concentrates on the single most-dangerous waypoint.
- $c \to 0$: sigmoid gate becomes a hard threshold — only $d_{ij} < 0$ obstacles contribute any gradient.
- $c \to \infty$: sigmoid gate $\to 0.5$ everywhere — recovers the unmodified softmin gradient (all obstacles contribute equally).

5 Generative Sampling with CBF and Diffusion PF-ODE

$$\frac{dx_t}{dt} = f(x_t, t) + \frac{g^2(t)}{2}\, s_\theta(x_t, t) \tag{37}$$

5.1 Control Affine Equation

$$\dot{x} = F(x,t) + G(x,t)u \tag{38}$$

where $F(x,t)$ is the drift and $G(x,t)$ is the control term. The PF-ODE can be represented as

$$\frac{dx_t}{dt} = f(x_t, t) + \frac{g^2(t)}{2}, s_\theta(x_t, t) + I \cdot u \tag{39}$$

$$\text{with } G(x) = I \text{ and } u = \vec{0} \text{ and } F(x) = f(x,t) + \frac{g^2(t)}{2} s_\theta(x,t) \tag{40}$$

$$\text{with } G(x) = I \text{ and } u = \vec{0} \text{ and } F(x) = f(x,t) + \frac{g^2(t)}{2\sigma_t} \epsilon_\theta(x,t) \tag{41}$$

So, for safe operation, QP objective and constraints can be given by

$$\text{minimize } \frac{1}{2}u_d^T Q u_d + c^T u_d + \frac{1}{2}\gamma\mu^2 \tag{42}$$

$$\text{S.T.} \tag{43}$$

$$-L_f h(x) - Lgh(x) \cdot u - \alpha(h(x)) - \mu h(x) \leq 0 \tag{44}$$

where

$$Q = I, \qquad c = \vec{0} \tag{45}$$

5.2 Optimal Control formulation using closed form solution of QP with CBF constraint:

$$u^*(x) = \begin{cases} -\dfrac{\omega(x)}{d(x)}\nabla h(x), & \omega(x) < 0 \\ 0, & \omega(x) \geq 0 \end{cases} \tag{46}$$

$$u^*(x) = -Q^{-1}\left(c - L_g h(x)^T \lambda(x)\right) \tag{47}$$

$$\lambda(x) = \begin{cases} \dfrac{-\omega(x)}{d(x)}, & x \in \Omega \\ 0, & x \notin \Omega \end{cases} \tag{48}$$

$$\omega(x) = L_f h(x) - L_g h(x) Q^{-1} c + \alpha(h(x)) \tag{49}$$

$$d(x) = L_g h(x) Q^{-1} L_g h(x)^T + \gamma^{-1} h(x)^2 \tag{50}$$

5.3 Substituting assumptions:

$$L_g h(x) = \nabla_x h(x) \cdot I \tag{51}$$

$$u^*(x) = \lambda(x)\, \nabla_x h(x) \tag{52}$$

$$\lambda(x) = \frac{-\omega(x)}{\|\nabla_x h(x)\|_2^2 + \gamma^{-1} h(x)^2} \tag{53}$$

5.4 Substituting VE ODE Drift:

$$L_f h(x) = \nabla_x h(x)^T \left(\frac{d \log \alpha_t}{dt} x - 2\sigma_t^2 \frac{d\lambda_t}{dt} \epsilon_\theta(x,t)\right) \tag{54}$$

$$\omega(x) = \nabla_x h(x)^T \left(\frac{d \log \alpha_t}{dt} x - 2\sigma_t^2 \frac{d\lambda_t}{dt} \epsilon_\theta(x,t)\right) + \alpha(h(x)) \tag{55}$$

5.5 Final Safe Control Formulation

$$u^*(x) = -\frac{\nabla_x h(x)^T \left(\frac{d\log\alpha_t}{dt}x - 2\sigma_t^2 \frac{d\lambda_t}{dt}\epsilon_\theta(x,t)\right) + \alpha(h(x))}{\|\nabla_x h(x)\|_2^2 + \gamma^{-1}h(x)^2}\,\nabla_x h(x) \tag{56}$$

Integrating

$$\dot{x} = f(x,t) + u^*(x) \tag{57}$$

using an exponential integrator of the form

$$x_t = e^{\int_s^t f(r)\,dr}\,x_s + \int_s^t e^{\int_\tau^t f(r)\,dr}\left[\frac{g^2(\tau)}{2\sigma_\tau}\epsilon_\theta(x_\tau,\tau) + u^*(x_\tau,\tau)\right]d\tau \tag{58}$$

where,

$$u^*(x_\tau,\tau) = \begin{cases} -\dfrac{\omega(x_\tau,\tau)}{\|\nabla h(x_\tau)\|_2^2 + \gamma^{-1}h(x_\tau)^2}\nabla h(x_\tau), & \omega(x_\tau,\tau) < 0, \\ 0, & \omega(x_\tau,\tau) \geq 0, \end{cases} \tag{59}$$

$$\omega(x_\tau,\tau) = \nabla h(x_\tau)^\top \left(f(\tau)x_\tau + \frac{g^2(\tau)}{2\sigma_\tau}\epsilon_\theta(x_\tau,\tau)\right) + \alpha(h(x_\tau)). \tag{60}$$

the sampling equation can be rewritten as

$$x_t = e^{\int_s^t f(r)\,dr}\,x_s + \int_s^t e^{\int_\tau^t f(r)\,dr}\left[\frac{g^2(\tau)}{2\sigma_\tau}\epsilon_\theta(x_\tau,\tau) - \frac{\min\Big(0,\ \nabla h(x_\tau)^\top\left(f(\tau)x_\tau + \frac{g^2(\tau)}{2\sigma_\tau}\epsilon_\theta(x_\tau,\tau)\right) + \alpha(h(x_\tau))\Big)}{\|\nabla h(x_\tau)\|_2^2 + \gamma^{-1}h(x_\tau)^2}\nabla h(x_\tau)\right]d\tau. \tag{61}$$

6 Safe DPM-Solver with CBF Control

6.1 Controlled Probability Flow ODE

The generative ODE with CBF safety control is:

$$\dot{x}_t = f(t)\,x_t + \frac{g^2(t)}{2\sigma_t}\,\epsilon_\theta(x_t,t) + u^*(x_t,t) \tag{62}$$

6.2 Exact Solution via Variation of Constants

Define the log-SNR $\lambda_t := \log(\alpha_t/\sigma_t)$. Multiplying through by the integrating factor $e^{-\int f(r)dr} = \alpha_t^{-1}$ and applying variation of constants:

$$x_t = \frac{\alpha_t}{\alpha_s}\,x_s + \alpha_t\int_s^t \frac{1}{\alpha_\tau}\left[\frac{g^2(\tau)}{2\sigma_\tau}\,\epsilon_\theta(x_\tau,\tau) + u^*(x_\tau,\tau)\right]d\tau \tag{63}$$

$$x_t = \frac{\alpha_t}{\alpha_s}\,x_s + \alpha_t\int_s^t \frac{1}{\alpha_\tau}\left[\frac{g^2(\tau)}{2\sigma_\tau}\,\epsilon_\theta(x_\tau,\tau)\right]d\tau + \alpha_t\int_s^t \frac{1}{\alpha_\tau}\left[u^*(x_\tau,\tau)\right]d\tau \tag{64}$$

Changing variable $\tau \to \lambda$ via $d\tau = d\lambda/\dot{\lambda}_\tau$ and using the identity from [**?**]:

$$\frac{g^2(\tau)}{2\sigma_\tau\,\alpha_\tau\,\dot{\lambda}_\tau} = -e^{-\lambda} \tag{65}$$

yields the exact solution in λ-space:

$$x_t = \frac{\alpha_t}{\alpha_s}\,x_s - \alpha_t\int_{\lambda_s}^{\lambda_t} e^{-\lambda}\,\epsilon_\theta(\hat{x}_\lambda,\lambda)\,d\lambda + \alpha_t\int_s^t \frac{u^*(x_\tau,\tau)}{\alpha_\tau}\,d\tau \tag{66}$$

6.3 Taylor Expansion of Nonlinear Terms

Denote $h_i := \lambda_{t_i} - \lambda_{t_{i-1}} > 0$. For $k = 1$ (DPM-Solver-1), approximate both ϵ_θ and u^* as constant over $[\lambda_s, \lambda_t]$:

Score integral (analytic, from [?]):

$$-\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda}\, \epsilon_\theta(x_s, s)\, d\lambda = -\alpha_t\, \epsilon_\theta(x_s, s) \left[-e^{-\lambda}\right]_{\lambda_s}^{\lambda_t} = -\sigma_t(e^{h_i} - 1)\, \epsilon_\theta(x_s, s) \tag{67}$$

CBF integral (constant approximation):

$$\alpha_t \int_s^t \frac{u^*(x_\tau, \tau)}{\alpha_\tau}\, d\tau = \frac{\alpha_t}{\alpha_s}\, u^*(x_s, s) \int_s^t d\tau = \frac{\alpha_t}{\alpha_s}(t - s) u^*(x_s, s) \tag{68}$$

6.4 CBF Optimal Control

With $g(x) = I$, $Q = I$, $c = 0$:

$$u^*(x_s, s) = -\frac{\min(0,\, \omega_s)}{\|\nabla h_s\|_2^2 + \gamma^{-1} h_s^2}\, \nabla h_s \tag{69}$$

where:

$$\omega_s = \nabla h_s^\top \left(f(s)\, x_s + \frac{g^2(s)}{2\sigma_s}\, \epsilon_\theta(x_s, s) \right) + \alpha(h_s) \tag{70}$$

6.5 Safe DPM-Solver-1 Update

Combining all terms:

$$x_{t_{i-1} \to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\, x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{71}$$

$$- \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}(t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top \left(f(t_{i-1})\, x_{t_{i-1}} + \frac{g^2(t_{i-1})}{2\sigma_{t_{i-1}}}\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \right) + \alpha(h_{t_{i-1}}) \right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1} h_{t_{i-1}}^2}\, \nabla h_{t_{i-1}} \tag{72}$$

$$x_{t_{i-1} \to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\, x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{73}$$

$$- \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}(t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top \left(f(t_{i-1})\, x_{t_{i-1}} + \frac{g^2(t_{i-1})}{2\sigma_{t_{i-1}}}\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \right) + \alpha(h_{t_{i-1}}) \right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1} h_{t_{i-1}}^2}\, \nabla h_{t_{i-1}} \tag{74}$$

For the VE Case, $\alpha_t = 1, f(t) = 0$ and $g^2(t) = \dot{\sigma}_{t_{i-1}}$

$$\boxed{x_{t_{i-1} \to t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) - (t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top \left(\dot{\sigma_{t_{i-1}}} \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) + \alpha(h_{t_{i-1}}) \right) \right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1} h_{t_{i-1}}^2}\, \nabla h_{t_{i-1}}} \tag{75}$$

$$\boxed{x_{t_{i-1} \to t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) - \dot{\sigma_{t_{i-1}}} * \frac{min\left(0, \nabla h_{t_{i-1}}^\top \left(\dot{\sigma_{t_{i-1}}} \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) + \alpha(h_{t_{i-1}}) \right) \right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1} h_{t_{i-1}}^2}\, \nabla h_{t_{i-1}}} \tag{76}$$

6.6 Properties

- **Generic:** Valid for any VE/VP schedule — only α_t, σ_t, $\dot{\lambda}_t$ need to be specified.
- **Recovery:** When $\omega_{t_{i-1}} \geq 0$, $\min(0, \omega) = 0$ and the update reduces exactly to standard DPM-Solver-1:

$$x_{t_{i-1} \to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{77}$$

- **Consistent order:** Both ϵ_θ and u^* are held constant at $x_{t_{i-1}}$, giving uniform $\mathcal{O}(h_i^2)$ truncation error.
- **No extra network calls:** The CBF correction reuses $\epsilon_\theta(x_{t_{i-1}}, t_{i-1})$ already computed for the score step.
- **Schedule coefficient:** $\dfrac{\alpha_{t_i}\, h_i}{\alpha_{t_{i-1}}\, \dot{\lambda}_{t_{i-1}}}$ rescales the CBF correction to match the log-SNR timescale, consistently with the $(e^{h_i} - 1)$ factor in the score term.

7 AG

(t) is a geometric schedule over discrete levels i = 0 . . . N:

$$\sigma_i = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^{i/N}$$

Default: $_m in = 0.01, _m ax = 10.0, N = 1000$.
The time derivative is:

$$\dot{\sigma}(t) = \sigma(t) \ln! \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)$$

Training Loss (Denoising Score Matching) Sample $x_0 \sim p_{\text{data}}$, $\varepsilon \sim \mathcal{N}(0, I)$, pick random level i:

$$\mathcal{L} = E\left[|\varepsilon_\theta(x_0 + \sigma_i \varepsilon,\ \sigma_i) - \varepsilon|^2 \right]$$

The network predicts the noise (not the score directly; score = /).
DPM-Solver-1 + CBF Update (from image) One denoising step $t_{i-1} \to t_i$ (decreasing):

$$x_{t_{i-1} \to t_i} = x_{t_{i-1}}(\sigma_{t_{i-1}} - \sigma_{t_i}), \varepsilon_\theta(x_{t_{i-1}}, t_{i-1}) \frac{h_i, \sigma_{t_{i-1}}}{\dot{\sigma}_{t_{i-1}}} \cdot \frac{\min(0, \omega_{t_{i-1}}^{\text{VE}})}{|\nabla h_{t_{i-1}}|2^2 + \gamma^{-1} ht_{i-1}{}^2}, \nabla h_{t_{i-1}}$$

where

$$h_i = \log! \left(\frac{\sigma_{t_{i-1}}}{\sigma_{t_i}} \right), \qquad \omega_s^{\text{VE}} = \dot{\sigma}_s, \nabla h_s^\top \varepsilon_\theta(x_s, s) + \alpha(h_s)$$

$h(x)$ — scalar CBF (positive in safe/free space, negative in walls). We use a differentiable signed-distance-to-walls function built from the maze grid. $\alpha(h)$ — class-K function; we use the linear $\alpha(h) = \alpha_0 \cdot h$ with $\alpha_0 = 1.0$. γ — regularisation parameter (default 1.0). The CBF term is zero whenever $\omega^{\text{VE}} \geq 0$ (constraint already satisfied).